Program Studi Teknik Informatika
Fakultas Teknologi Industri, Institut Teknologi Sumatera

IF25-11002 PRAKTIKUM PEMROGRAMAN

Modul Belajar 11

Searching

Mencari data di array dengan linear search dan binary search, dan membandingkan banyak langkahnya

Tim Pengajar
Pendamping Pertemuan 11. Tahun 2026

Komponen	Keterangan
Pertemuan	11 dari 16
Capaian	CPMK0607 (menerapkan) dan CPMK0608 (menjelaskan) pada konsep dasar algoritma dan sistem komputer
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P11 - Searching - Hands-on/`

MODUL 11

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: Mencari nama di daftar wisuda
3	Langkah 1: Linear search
4	Langkah 2: Mencari semua kemunculan
5	Langkah 3: Ide binary search
6	Langkah 4: Binary search di C++
7	Langkah 5: Berapa langkahnya?
8	Pesan error yang sering muncul
9	Latihan mandiri
10	Rangkuman
11	Cek pemahaman
12	Exit Ticket dan tugas
13	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini mendampingi Pertemuan 11. Sejauh ini kalian sudah menyimpan banyak data di array. Pertanyaan berikutnya: bagaimana menemukan satu data di antaranya dengan cepat? Minggu ini kalian membandingkan dua cara mencari, dan melihat mengapa data yang terurut bisa dicari jauh lebih cepat.

Isinya sama dengan slide di kelas, tetapi ditulis lebih pelan dan lebih lengkap, supaya kalian bisa

mengulang sendiri di rumah tanpa harus menebak maksud slide.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menulis program yang mencari data pada array dengan linear search dan binary search, termasuk mencari semua kemunculan dan menangani data yang tidak ditemukan (Sub-CPMK 11.1, CPMK0607)	Langkah 1 sampai 5
2	Menelusuri langkah binary search dengan trace table low, high, dan mid, serta menjelaskan perbedaan banyak perbandingan kedua algoritma (Sub-CPMK 11.2, CPMK0608)	kotak Tebak dulu, Latihan 3

Karena setiap instrumen penilaian dibagi rata antara Bagian A (menulis kode) dan Bagian B (menelusuri dan menjelaskan), yang dinilai bukan hanya program kalian jalan, tetapi juga kalian bisa menjelaskan mengapa program itu jalan.

Cara memakai modul ini

Setiap langkah punya pola yang sama: penjelasan konsep, kadang contoh dari kehidupan sehari-hari, lalu kode yang bisa langsung kalian ketik. Sesudah kode ada kotak **Tebak dulu**. Tulis tebakan kalian di kertas sebelum membaca **Jawaban** di bawahnya. Kebiasaan menebak lalu membuktikan inilah yang membuat konsep menempel, bukan sekadar membaca.

> PENTING **Ketik, jangan salin**
> Ketik ulang setiap contoh di GDB Online atau editor kalian, jangan disalin-tempel. Saat mengetik, kalian akan membuat salah ketik, membaca pesan error, dan memperbaikinya. Itu bagian dari belajar.

Yang perlu disiapkan

Peramban dengan akses ke onlinegdb.com, atau g++ di komputer kalian sendiri. Kertas dan alat tulis untuk menebak keluaran dan membuat trace table. Folder hands-on pertemuan ini dari repo mata kuliah.

PEMBUKA

Masalah pembuka: Mencari nama di daftar wisuda

Panitia wisuda memegang daftar 1.200 nama yang sudah urut abjad. Seorang orang tua bertanya, apakah anaknya ada di daftar?

Membaca dari nama pertama sampai ketemu bisa butuh ratusan kali melihat. Padahal saat mencari kata di kamus, kita tidak membaca dari halaman pertama.

Tampilan yang diinginkan

```
Cari: Siti Rahma
Ditemukan di nomor 1023
Linear search : 1023 perbandingan
Binary search : 10 perbandingan
```

TEBAK DULU **Coba pikirkan**

Bagaimana kalian mencari kata di kamus? Mengapa cara itu tidak bisa dipakai pada daftar yang belum urut?

JAWABAN **Cara memecahnya**

Bandingkan satu per satu: Cara paling sederhana: periksa setiap elemen dari depan. Manfaatkan urutan: Bila data urut, buang separuh daftar di setiap langkah. Terjemahkan: Linear search dengan `for`, binary search dengan `while` dan tiga indeks.

LANGKAH 1

Linear search

Linear search memeriksa elemen satu per satu dari depan. Begitu cocok, posisinya dicatat dan pencarian berhenti dengan `break`. Bila sampai akhir tidak ada yang cocok, variabel posisi tetap -1.

Linear search bekerja pada data apa pun, terurut atau tidak. Kelemahannya, pada kasus terburuk semua elemen diperiksa.

> ANALOGI **Contoh sehari-hari**
>
> Mencari kunci motor di tas yang berantakan: kalian mengeluarkan barang satu per satu sampai kuncinya ketemu, atau sampai tas kosong.

Bentuk umum

```
int posisi = -1;
for (int i = 0; i < n; i++)
    if (a[i] == x) { posisi = i; break; }
```

linear.cpp

```
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 7, posisi = -1;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        posisi = i;
        break;
    }
}
if (posisi == -1) cout << "tidak ada" << endl;
else cout << "indeks " << posisi << endl;
```

> TEBAK DULU
>
> Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

> JAWABAN

Keluaran

```
indeks 3
```

Nilai awal -1 menandai belum ditemukan, karena -1 tidak pernah menjadi indeks yang sah.

LANGKAH 2

Mencari semua kemunculan

Kadang yang ditanyakan bukan di mana pertama kali, tetapi di mana saja dan berapa kali. Untuk itu `break` dihilangkan dan sebuah pencacah ditambahkan.

ANALOGI **Contoh sehari-hari**

Seperti mencari semua teman bernama Rani di daftar hadir: kita membaca sampai habis dan mencatat setiap yang cocok.

semua.cpp

```cpp
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 9, banyak = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        cout << "indeks " << i << endl;
        banyak++;
    }
}
cout << "muncul " << banyak << " kali" << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
indeks 1
indeks 5
muncul 2 kali
```

Untuk semua kemunculan, perulangan tidak dihentikan; pencacah mencatat banyaknya.

LANGKAH 3

Ide binary search

Bila data sudah terurut, kita tidak perlu memeriksa dari depan. Periksa elemen tengah. Bila yang dicari lebih kecil, buang separuh kanan; bila lebih besar, buang separuh kiri. Setiap langkah membuang separuh sisa data.

Tiga indeks menjaga batas: `low` dan `high` mengapit bagian yang masih mungkin, `mid` adalah tengahnya. Trace table berikut menunjukkan pencarian 21.

ANALOGI **Contoh sehari-hari**

Tebak angka 1 sampai 100. Penebak yang cerdik selalu menebak tengah, lalu membuang separuh kemungkinan setiap kali diberi tahu terlalu besar atau terlalu kecil.

Langkah	low	high	mid	a[mid]	Keputusan
1	0	9	4	34	34 > 21, cari di kiri: high = 3
2	0	3	1	8	8 < 21, cari di kanan: low = 2
3	2	3	2	15	15 < 21, low = 3
4	3	3	3	21	ditemukan di indeks 3

PENTING

Binary search hanya benar pada data yang sudah terurut.

LANGKAH 4

Binary search di C++

Terjemahan langsung dari trace table. Perhatikan tiga detail yang sering salah: syarat `low <= high`, pergeseran `mid + 1` dan `mid - 1` (bukan `mid`), dan arah pergeseran yang mengikuti hasil perbandingan.

Bentuk umum

```
int low = 0, high = n - 1;
while (low <= high) {
    int mid = (low + high) / 2;
    // bandingkan a[mid] dengan x
}
```

biner.cpp

```
int a[10] = {3, 8, 15, 21, 34, 42, 57, 66, 79, 90};
int x = 21, low = 0, high = 9, posisi = -1;
while (low <= high) {
    int mid = (low + high) / 2;
    if (a[mid] == x) { posisi = mid; break; }
    else if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
}
cout << "indeks " << posisi << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
indeks 3
```

Syaratnya `low <= high`. Dengan <, elemen terakhir yang tersisa tidak sempat diperiksa.

LANGKAH 5

Berapa langkahnya?

Pada kasus terburuk, linear search memeriksa semua n elemen. Binary search hanya butuh sekitar $\log_2 n$ langkah, karena data terbagi dua di setiap langkah. Selisihnya makin besar ketika datanya makin banyak.

> ANALOGI **Contoh sehari-hari**
>
> Seperti mencari nama di buku telepon setebal 1.000 halaman: membaca halaman per halaman bisa butuh 1.000 langkah, membuka di tengah lalu membelah dua hanya sekitar 10.

Banyak data	Linear, kasus terburuk	Binary, kasus terburuk
10	10	4
100	100	7
1.000	1.000	10
1.000.000	1.000.000	20

> PENTING
>
> Binary search jauh lebih cepat, tetapi butuh data terurut. Mengurutkan data adalah materi minggu depan.

WASPADA

Pesan error yang sering muncul

Semua pesan di tabel ini dihasilkan g++ dengan opsi `-Wall`, sama seperti yang dipakai `cek.sh`.

Kesalahan	Contoh	Pesan pertama dari g++
Membandingkan array dengan ==	`if (a == x)`	`error: ISO C++ forbids comparison between pointer and integer`
Variabel di luar jangkauan	`mid dipakai di luar while`	`error: 'mid' was not declared in this scope`
Kutip tunggal untuk string	`nama[i] == 'Rani'`	`error: no match for 'operator=='`
Posisi tidak dipakai	`int posisi = -1; tak dipakai`	`warning: unused variable 'posisi'`
Kurang titik koma di dalam if	`posisi = mid break;`	`error: expected ';' before 'break'`

Kesalahan binary search yang paling sering, yaitu syarat `low < high` dan arah pergeseran yang terbalik, tidak memunculkan pesan apa pun. Program tetap jalan tetapi jawabannya salah. Trace table adalah satu-satunya cara cepat menemukannya.

LATIHAN

Latihan mandiri

Latihan ini sama dengan hands-on di kelas. Semua file ada di `02 Hands-on/P11 - Searching - Hands-on/latihan/`. Kerjakan berurutan, lalu cocokkan dengan `expected/` atau jalankan `bash cek.sh`.

No	File	Tingkat	Yang dilatih
1	`handson1-cari.cpp`	Dasar	linear search, hasil -1, hitung perbandingan
2	`handson2-nama.cpp`	Menengah	mencari semua kemunculan string
3	`handson3-biner.cpp`	Tantangan	trace low-high-mid, perbaiki tiga kesalahan
+	`bonus-banding.cpp`	Pengayaan	menghitung perbandingan dua algoritma

Latihan 1: handson1-cari.cpp

Linear search dengan kasus ditemukan dan tidak ditemukan, serta banyak perbandingan.

Latihan 2: handson2-nama.cpp

Cari semua posisi sebuah nama di daftar hadir dan banyak kemunculannya.

Latihan 3: handson3-biner.cpp

Binary search dengan tiga kesalahan. Buat trace table untuk dua pencarian, perbaiki, dan jelaskan.

Bonus: bonus-banding.cpp

Bandingkan banyak perbandingan linear dan binary search untuk empat nilai.

Latihan menelusuri

tebak.cpp

```
int a[7] = {2, 5, 8, 12, 16, 23, 38};
int x = 23, low = 0, high = 6, langkah = 0;
while (low <= high) {
    int mid = (low + high) / 2;
    langkah++;
    if (a[mid] == x) break;
    if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
```

```
}
cout << langkah << " " << low << " " << high << endl;
```

TEBAK DULU

Tulis keluarannya di kertas tanpa menjalankan program. Buat trace table bila perlu.

JAWABAN

Keluaran

```
2 4 6
```

mid = 3 (12 < 23, low = 4), lalu mid = 5 (23, ketemu). Dua langkah; saat berhenti low = 4 dan high = 6.

PENUTUP

Rangkuman

Konsep	Yang perlu diingat
Linear search	Nilai awal -1 menandai belum ditemukan, karena -1 tidak pernah menjadi indeks yang sah.
Mencari semua kemunculan	Untuk semua kemunculan, perulangan tidak dihentikan; pencacah mencatat banyaknya.
Ide binary search	Binary search hanya benar pada data yang sudah terurut.
Binary search di C++	Syaratnya `low <= high`.
Berapa langkahnya?	Binary search jauh lebih cepat, tetapi butuh data terurut.

PENUTUP

Cek pemahaman

Jawab tanpa menjalankan program, lalu periksa dengan menjalankannya.

1. Mengapa posisi diawali -1?
2. Berapa perbandingan linear search untuk mencari elemen terakhir dari 50 data?
3. Apa prasyarat binary search?
4. Mengapa syaratnya `low <= high`, bukan `low < high`?
5. Kira-kira berapa langkah binary search untuk 1.000 data?

> JAWABAN
>
> 1. -1 tidak pernah menjadi indeks yang sah, jadi menandai belum ditemukan.
> 2. 50.
> 3. Data harus sudah terurut.
> 4. Saat low sama dengan high masih ada satu elemen yang harus diperiksa.
> 5. Sekitar 10 langkah.

PENILAIAN

Exit Ticket 11

Dikerjakan di 25 menit terakhir kelas, sendiri, tanpa AI, internet, atau diskusi. Exit Ticket 11 adalah satu dari 14 Exit Ticket; rata-ratanya menjadi komponen berbobot 25%.

Soal	Isi	Poin	CPMK
A1	Tulis linear search yang menampilkan semua indeks kemunculan sebuah nilai	25	CPMK0607
A2	Tulis binary search pada array terurut yang menampilkan indeks atau tidak ditemukan	25	CPMK0607
B1	Isi trace table low, high, mid untuk sebuah pencarian	20	CPMK0608
B2	Jelaskan mengapa binary search gagal pada data yang belum terurut	20	CPMK0608
B3	Bandingkan banyak perbandingan dua algoritma untuk data yang diberikan	10	CPMK0608

PENILAIAN

Tugas 11: Direktori Kontak

Kumpulkan sebelum Pertemuan 12 lewat kanal pengumpulan yang diumumkan dosen.

Bagian A: program (50 poin)

Buat program direktori kontak dengan menu berikut. Data dimasukkan dalam urutan abjad nama; program menolak nama yang tidak urut.

1. Tambah kontak, maksimal 10 (nama satu kata dan nomor telepon), dengan validasi urutan abjad
2. Tampilkan semua kontak
3. Cari nama dengan linear search; tampilkan nomor dan banyak perbandingan
4. Cari nama dengan binary search; tampilkan nomor dan banyak perbandingan
5. Keluar

> ANALOGI **Bonus, tidak wajib**
>
> Tambahkan pencarian awalan nama, misalnya semua kontak yang diawali ”Ra”.

Bagian B: penjelasan (50 poin)

Komentar maksimal 150 kata: trace table binary search untuk satu nama yang ada dan satu yang tidak ada, serta perbandingan banyak langkah linear dan binary search pada data kalian. Jelaskan mengapa data harus urut abjad.

Rubrik Tugas 11

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Kelengkapan menu	15	lima menu dan validasi urutan	satu menu kurang	dua kurang	tanpa binary search
A3 Ketepatan pencarian	15	kedua pencarian tepat, termasuk tidak ditemukan	satu kasus salah	dua salah	pencarian salah
B1 Trace table dan perbandingan	30	dua trace tepat dan analisis jelas	satu trace keliru	tanpa analisis	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Contoh: A1 di level Baik dengan skor 80 memberi 20 × 0,80 = 16 poin.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

1. Deitel, P. dan Deitel, H. C++ How to Program, edisi ke-10, Bab 8 (searching).
2. learncpp.com, Bab 18 (algoritma pada array).
3. Visualgo.net, modul Searching dan Sorting.
4. Slide `01 Slide/P11 Searching.pdf` dan hands-on di `02 Hands-on/P11 - Searching - Hands-on/`.

> PENTING **Minggu depan**
> Pertemuan 12 membahas sorting: bubble sort dan selection sort, supaya data bisa diurutkan sebelum dicari dengan binary search.